

TP N°1 : Classe

Il s'agit d'écrire une classe *CPoint* permettant de gérer des points représentés sur le plan cartésien par des coordonnées entières.

1. Démarrez Visual C++. Créez un nouveau projet *Win32 Console Application* (Fichier/Nouveau/Projet → Projet Visual C++ (Type de projet) → Projet Win 32 (Modèle)) nommé *MonPoint*, dans le répertoire *C:\Temp\Votre Nom\TP1*, sous la forme d'une *Application Console* et d'un *projet vide* à modifier dans les paramètres de l'application.

Note : Cette étape devra être utilisée pour chaque TP, merci de l'approfondir.

2. Créez la classe principale *CPoint* à l'aide du *Browser* (cliquez sur l'icône *Affichage de Classes*, ensuite cliquez à l'aide du bouton droit sur le nom du projet (*MonPoint*), faire *Ajouter une classe*, etc.).
3. Complétez votre classe avec les données et les méthodes membres suivants :
 - en membres données privées : deux composantes *m_x* et *m_y* de type entier,
 - deux constructeurs (fonctions) :
 - *CPoint()*, sans arguments, initialisant chaque composante (x et y) à 0,
 - *CPoint(int, int)*, avec 2 arguments, qui permettront d'initialiser les composantes *m_x* et *m_y*,
 - un destructeur *~CPoint()*.

Note : Aidez vous du browser, cliquez sur l'icône *Affichage de Classes*, ensuite cliquez à l'aide du bouton droit sur le nom de la classe (*CPoint*).

m_ : indique à l'utilisateur (non au compilateur) que la donnée est membre de la classe (*member variable*)

4. Créez un fichier *main.cpp* à l'aide du *Browser* (cliquez sur l'icône *Affichage de Fichiers*, ensuite cliquez à l'aide du bouton droit sur le nom du projet (*MonPoint*), faire *Ajouter* → *Ajouter un nouvel élément* → *Fichier C++*, etc.).

Dans votre programme principal *void main (void)* déclarez un objet instance automatique de type *CPoint*. Compilez, exécutez.

5. Ajouter à votre constructeur sans paramètres l'instruction : *cout << "Passage dans le constructeur sans paramètres" << endl ;*
Dans votre fonction *main()*, déclarez un objet *toto* de type *CPoint*. Exécutez. Expliquez les messages obtenus à l'écran.
6. Ajouter à votre destructeur l'instruction : *cout << "Passage dans le destructeur" << endl ;*
7. Ajouter à votre constructeur avec paramètres l'instruction : *cout << "Passage dans le constructeur avec paramètres" << endl ;*
Dans votre fonction *main()*, déclarez un objet *lili* de type *CPoint* avec les paramètres 5 et 10 par exemple. Exécutez. Expliquez les messages obtenus à l'écran.
8. Ajoutez à la classe *CPoint* une fonction membre nommée *void Affiche()* permettant d'afficher la position d'un point donné. Testez la fonction *Affiche()* dans le programme principal.
Note : Aidez vous du browser.
9. Ajoutez à la classe *CPoint* une fonction membre nommée *void Translate()* permettant de faire la translation d'un point (x, y) d'un déplacement (dx, dy). La fonction *Translate()* aura deux arguments entiers *dx* et *dy*. Testez votre fonction dans le programme principal.
10. Dans votre fonction *main()*, déclarez deux objets *pt1(5, 2)* et *pt2(2, 3)* de type *CPoint*. Affichez les ensuite.

11. Modifiez le corps de votre fonction *main()* afin de traduire de (2, 3) votre objet *pt1*. Affichez sa nouvelle position.
12. Dans votre fonction *main()* affecter à la composante *m_x* de l'objet *pt1* la valeur 10. Exécutez votre programme. Que se passe-t-il ? Pourquoi ?
13. Est-il permis de faire les opérations suivantes ?
 - *pt1 = pt2*,
 - *pt1.m_x = pt2.m_x, pt1.m_y = pt2.m_y*, Expliquez.
14. Ajouter à la classe *CPoint* un constructeur **inline**, *CPoint(int)*, avec un seul argument permettant d'initialiser les 2 composantes *m_x* et *m_y* à la même valeur. Tester ce constructeur dans votre programme principal.
15. Ajouter à la classe *CPoint* une fonction *bool Coïncide()* permettant de vérifier si 2 points donnés coïncident. Cette fonction aura comme argument un objet de type *CPoint* et enverra en sortie un booléen de valeur VRAI ou FAUX. Tester cette fonction dans votre fonction *main()*.

LE TP DOIT ETRE FINI POUR LA SEANCE SUIVANTE